

LiDAR Guard

LiDAR Guard

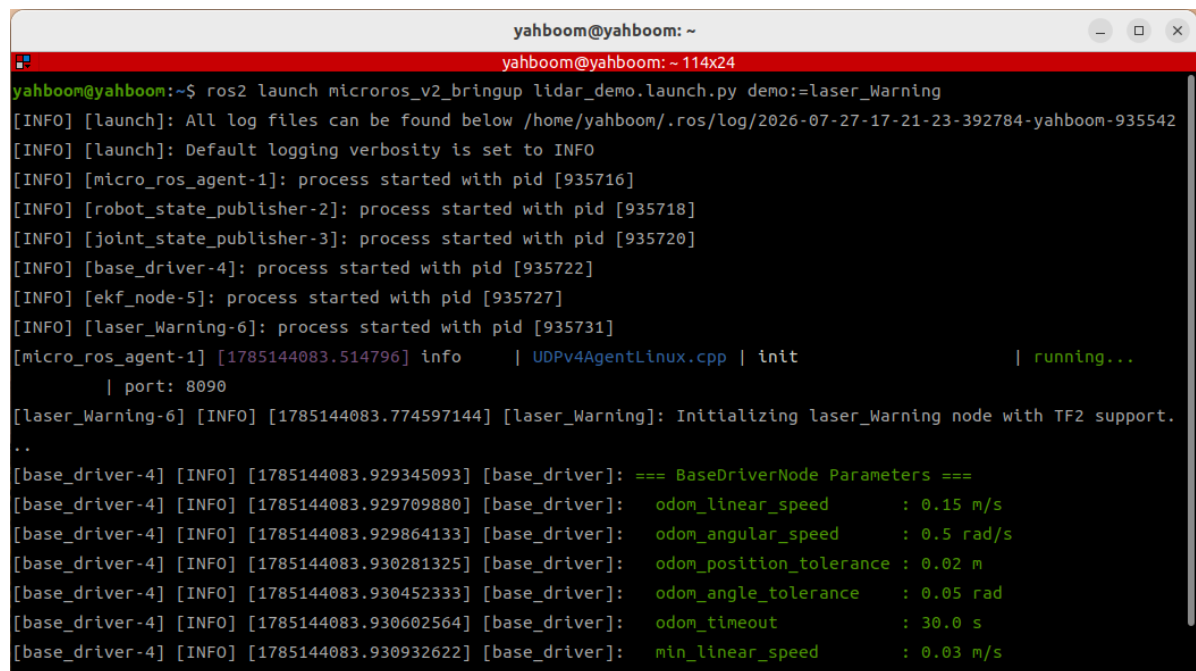
1. Course Content
2. Demo
3. Source Code Analysis

1. Course Content

- Run the demo program. It will detect in real time the object closest to the robot body and control the chassis to rotate in place to track it. At the same time, if the distance between the object and the robot is smaller than the alarm threshold, the buzzer on the robot will sound an alarm.

2. Demo

```
ros2 launch microros_v2_bringup lidar_demo.launch.py demo:=laser_warning
```



```
yahboom@yahboom: ~  
yahboom@yahboom: ~ 114x24  
yahboom@yahboom:~$ ros2 launch microros_v2_bringup lidar_demo.launch.py demo:=laser_warning  
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-07-27-17-21-23-392784-yahboom-935542  
[INFO] [launch]: Default logging verbosity is set to INFO  
[INFO] [micro_ros_agent-1]: process started with pid [935716]  
[INFO] [robot_state_publisher-2]: process started with pid [935718]  
[INFO] [joint_state_publisher-3]: process started with pid [935720]  
[INFO] [base_driver-4]: process started with pid [935722]  
[INFO] [ekf_node-5]: process started with pid [935727]  
[INFO] [laser_warning-6]: process started with pid [935731]  
[micro_ros_agent-1] [1785144083.514796] info      | UDPv4AgentLinux.cpp | init          | running...  
| port: 8090  
[laser_warning-6] [INFO] [1785144083.774597144] [laser_warning]: Initializing laser_warning node with TF2 support.  
..  
[base_driver-4] [INFO] [1785144083.929345093] [base_driver]: === BaseDriverNode Parameters ===  
[base_driver-4] [INFO] [1785144083.929709880] [base_driver]:   odom_linear_speed      : 0.15 m/s  
[base_driver-4] [INFO] [1785144083.929864133] [base_driver]:   odom_angular_speed     : 0.5 rad/s  
[base_driver-4] [INFO] [1785144083.930281325] [base_driver]:   odom_position_tolerance : 0.02 m  
[base_driver-4] [INFO] [1785144083.930452333] [base_driver]:   odom_angle_tolerance   : 0.05 rad  
[base_driver-4] [INFO] [1785144083.930602564] [base_driver]:   odom_timeout           : 30.0 s  
[base_driver-4] [INFO] [1785144083.930932622] [base_driver]:   min_linear_speed       : 0.03 m/s
```

3. Source Code Analysis

Source code path:

```
$HOME/microros_ws/src/common_demo/common_demo/lidar/laser_warning.py
```

Core logic of the guard node: **nearest obstacle detection** → **buzzer alarm** → **PID steering tracking**.

1. Nearest obstacle detection:

- Iterate over all laser points and filter out invalid values (≤ 0 or beyond `range_max`)

- Convert each point from polar to Cartesian coordinates and transform it to the `base_link` frame
- Keep only front points whose angle is within $\pm$ `LaserAngle`
- Select the closest obstacle (`min_dist`, `nearest_x`, `nearest_y`, `nearest_angle`) from these points
- If no obstacle is detected, publish zero velocity and turn off the buzzer

2. PID steering tracking:

- **Angular velocity PID:** With 0 (alignment) as the setpoint and `nearest_angle` (in radians) as the feedback, `ang_pid` computes the steering speed
- **Angular deadzone:** When `|ang_pid_output| < angular_deadzone`, output 0 to avoid wheel jitter caused by tiny deviations (note: `angular_deadzone` is not explicitly defined in the parameter file; the code default value is used)
- **Angular clamp:** When exceeding `max_angular`, clamp to $\pm$ `max_angular` to prevent overly aggressive steering
- Linear velocity is fixed at 0 (guard mode rotates in place and does not move forward)

```
def registerScan(self, scan_data):
    if not isinstance(scan_data, LaserScan): return

    # Auto-detect laser frame_id
    if not self.laser_frame and scan_data.header.frame_id:
        self.laser_frame = scan_data.header.frame_id
        self.get_logger().info(f'Detected laser frame: {self.laser_frame}')

    # Check if TF is ready
    if not self.tf_ready or not self.laser_frame:
        return

    # Check if transform is still available
    try:
        trans = self.tf_buffer.lookup_transform(
            'base_link',
            self.laser_frame,
            rclpy.time.Time()
        )
    except TransformException as ex:
        current_time = self.get_clock().now().nanoseconds / 1e9
        if (current_time - self.last_log_time) >= 3.0:
            self.get_logger().warn(f'TF transform error: {ex}')
            self.last_log_time = current_time
        return

    # Get laser data
    ranges = np.array(scan_data.ranges)

    # Find the nearest obstacle in front using TF2
    min_dist = float('inf')
    nearest_x = 0.0
    nearest_y = 0.0
    nearest_angle = 0.0
    found_obstacle = False

    for i in range(len(ranges)):
```

```

# Skip invalid measurements
if ranges[i] <= 0.0 or ranges[i] > scan_data.range_max:
    continue

# Calculate laser point position in laser coordinate frame
angle = scan_data.angle_min + scan_data.angle_increment * i
point_in_laser = PointStamped()
point_in_laser.header = scan_data.header
point_in_laser.point.x = ranges[i] * math.cos(angle)
point_in_laser.point.y = ranges[i] * math.sin(angle)
point_in_laser.point.z = 0.0

# Transform to base_link coordinate frame
try:
    point_in_base = do_transform_point(point_in_laser, trans)
    x = point_in_base.point.x
    y = point_in_base.point.y

    # Check if point is within LaserAngle in front
    dist = math.sqrt(x**2 + y**2)
    angle_to_base = math.atan2(y, x) * RAD2DEG

    if abs(angle_to_base) < self.LaserAngle:
        # Find nearest obstacle
        if dist < min_dist:
            min_dist = dist
            nearest_x = x
            nearest_y = y
            nearest_angle = angle_to_base
            found_obstacle = True
except Exception as e:
    continue

if not found_obstacle:
    # No obstacle found, stop robot
    velocity = Twist()
    self.pub_vel.publish(velocity)
    # Turn off buzzer
    if self.buzzer_active:
        buzzer_msg = UInt16()
        buzzer_msg.data = 0
        self.pub_Buzzer.publish(buzzer_msg)
        self.buzzer_active = False
    return

# Log nearest obstacle info (throttled)
current_time = self.get_clock().now().nanoseconds / 1e9
if (current_time - self.last_log_time) >= self.log_interval:
    self.get_logger().info(
        f'Nearest: dist={min_dist:.2f}m, '
        f'x={nearest_x:.2f}m, y={nearest_y:.2f}m, '
        f'angle={nearest_angle:.1f}°'
    )
    self.last_log_time = current_time

# Control buzzer based on distance
if min_dist <= self.ResponseDist:
    if not self.buzzer_active:

```

```

        buzzer_msg = UInt16()
        buzzer_msg.data = 1
        self.pub_Buzzer.publish(buzzer_msg)
        self.buzzer_active = True
    else:
        if self.buzzer_active:
            buzzer_msg = UInt16()
            buzzer_msg.data = 0
            self.pub_Buzzer.publish(buzzer_msg)
            self.buzzer_active = False

    # Calculate angular velocity to track the nearest obstacle
    # Target angle is 0 (aligned with x-axis)
    angle_rad = math.radians(nearest_angle)
    ang_pid_compute = self.ang_pid.pid_compute(0, angle_rad)

    velocity = Twist()

    # Reverse angular velocity direction
    velocity.angular.z = -ang_pid_compute

    # Apply deadzone
    if abs(ang_pid_compute) < self.angular_deadzone:
        velocity.angular.z = 0.0

    # Clamp angular velocity
    if abs(velocity.angular.z) > self.max_angular:
        velocity.angular.z = self.max_angular * (1.0 if velocity.angular.z >
0 else -1.0)

    # Publish velocity
    self.pub_vel.publish(velocity)

```

- Parameter file path

```
$HOME/microros_ws/src/microros_v2_bringup/config/common.yaml
```

Parameter descriptions for the `laser_warning` node in `common.yaml`:

Parameter	Default value	Description	Tuning suggestions
<code>linear</code>	0.15 m/s	Linear velocity (kept for guard mode, not actually used)	Guard mode rotates in place, so this parameter has no effect
<code>angular</code>	1.0 rad/s	Steering angular velocity (reserved parameter; actually determined by the PID output)	Guard mode is controlled by a PID closed loop, so this parameter does not take effect directly
<code>LaserAngle</code>	45.0°	Half-angle of the front warning sector	The larger the angle, the wider the search range, but lateral interference may be introduced
<code>ResponseDist</code>	0.55 m	Buzzer alarm distance	The smaller the distance, the more sensitive the alarm; adjust according to the guard scenario
<code>ang_pid_P/I/D</code>	3.0/0.0/5.0	Angular velocity PID parameters	P determines the steering response speed, D suppresses overshoot; recommended to start with P:D=3:5
<code>max_angular</code>	2.0 rad/s	Angular velocity limit	Limits the maximum steering speed to prevent overly aggressive turns

```

laser_warning:
  ros__parameters:
    linear: 0.15                # Linear speed (m/s), reserved parameter in
guard mode
    angular: 1.0                # Steering angular velocity (rad/s)
    LaserAngle: 45.0            # Half-angle of the front warning sector
(deg)
    ResponseDist: 0.55          # Buzzer alarm distance (m)

    # ---- Angular velocity PID (steering toward the obstacle) ----
    ang_pid_P: 3.0              # Angular velocity PID: proportional
    ang_pid_I: 0.0              # Angular velocity PID: integral
    ang_pid_D: 5.0              # Angular velocity PID: derivative

    # ---- Angular velocity deadzone and limiting ----
    max_angular: 2.0            # Angular velocity limit (rad/s)

```

